

Étude de cas — énoncé à distribuer aux étudiants

Contexte

Une enseigne reçoit ses commandes clients sous forme de lignes de texte brutes, simulant un export CSV simplifié. Chaque ligne suit le format :

```
id;produit;quantite;prixUnitaire
```

Exemple de jeu de données :

```
CMD001;Clavier;3;29.90
CMD002;Souris;-1;9.90
CMD003;;2;15.00
CMD004;Ecran;1;149.99
CMD005;Casque;deux;39.90
CMD006;Cle USB;5;7.5
```

Certaines lignes sont invalides : quantité négative ou non numérique, produit vide, prix non numérique ou nul. L'objectif est de construire un pipeline qui transforme chaque ligne soit en une commande valide, soit en un message d'erreur explicite — sans jamais lever d'exception.

Travail demandé

Ouvrez le projet pipeline-commandes-etudiant fourni. Complétez les fonctions marquées ??? // TODO dans src/main/scala/Pipeline.scala, dans l'ordre suivant :

1. `parseIntOpt` et `parseDoubleOpt` — parser un champ texte vers un nombre en utilisant `Option` (indice : `toIntOption` / `toDoubleOption`).
2. `validerProduit` — `Either[String, String]` : le produit ne doit pas être vide.
3. `validerQuantite` — `Either[String, Int]` : la quantité doit être un entier strictement positif. Combinez le parsing (`Option`) et la règle métier avec `toRight` puis `flatMap`.
4. `validerPrix` — `Either[String, Double]` : le prix doit être strictement positif.
5. `parseCommande` — assemblez les trois validations dans un `for-comprehension` qui produit un `Either[String, Commande]`.
6. `traiterLignes` — appliquez `parseCommande` à une liste de lignes avec `map`.
7. `resumerResultat` — utilisez `fold` pour produire un message "[OK] ..." ou "[ERREUR] ..." selon le résultat.
8. `calculerStatistiques` — calculez le nombre de commandes valides, le nombre d'erreurs et le montant total des commandes valides.

Livrables attendus

- Le fichier `Pipeline.scala` complété, qui compile sans erreur (`sbt compile`).
- Une exécution `sbt run` affichant un message par ligne du jeu de données, puis un récapitulatif des statistiques.
- Si le temps le permet, les tests fournis dans `PipelineSpec.scala` passent (`sbt test`).

Consignes d'organisation

- Travail individuel.
- Le squelette de projet doit être utilisé tel quel : ne pas modifier les signatures des fonctions ni le nom des case class.